

Real-world (act out a canary release) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real canary rollout decides whether to trust the new version). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A canary release means most requests keep going to the old, trusted version (v1), while a small slice — say 1 in 5 — go to the new version (v2) instead. You don't decide this per-batch, you decide it per-request, randomly, as each one comes in.

Things to use:

- `random.random()` — gives a random number between 0.0 and 1.0.
- `if random.random() < 0.2:` — true about 20% of the time, on average.
- A list to record which version handled each request, and what it scored.
- Simple math at the end (`sum()`, dividing by count) to compare the two groups.

Intermediate Version

The whole trick of a canary release is that the *routing decision* happens independently for every single request — you're not splitting your list of test requests into "the first 20%" and "the rest," which would just be an arbitrary chunk, not a genuine random sample. `random.random() < 0.2` called once per request gives you a fair, independent coin flip each time, so the canary group ends up looking statistically like a real slice of your traffic, not whatever happened to be first in the file.

You need somewhere to record the outcome of each request, tagged with which version handled it, so you can group and compare afterward — a list of small dicts is the natural fit here, the same shape you've used for the version log.

The exact pieces:

- `import random` — `random.random()` returns a float in `[0.0, 1.0)`.
- A loop over your test requests: `for request in test_requests:`
- A results list: `append {"version": "v1" or "v2", "score": ..., "cost": ...}` for every request, inside the loop.
- After the loop: separate the results by version (two lists, built with a plain `for` loop and `if`, not a comprehension), then `sum(...)` / `len(...)` for each group's average score and average cost.

Advanced Version

Think past "route randomly and print two averages" — a real canary release exists to answer one question: **is it safe to give v2 the rest of the traffic?** That means you need a *decision*, not just two numbers sitting side by side for a human to eyeball. Set an explicit rule up front — for example, "the canary group's average score must be at least as good as the control group's, and its average cost must not be more than 20% higher" — and have your script actually apply that rule and print a verdict.

The other real gap in the Basic/Intermediate approach: 20% of a *small* test set (say, 10 requests) is 2 requests — nowhere near enough to trust an average. A real canary rollout either uses a large enough sample, or explicitly flags when the canary group is too small to draw a conclusion from yet, instead of confidently reporting a verdict built on 2 data points.

The extra pieces needed:

- A small `evaluate_canary(control_results, canary_results, min_sample_size)` function that returns a verdict (`"promote"` , `"hold"` , or `"insufficient data"`), applying explicit, named thresholds rather than eyeballing two printed numbers.
- A sample-size guard: `if len(canary_results) < min_sample_size: return "insufficient data"` , checked *before* any score comparison.
- Report the *gap*, not just each group's raw number — `canary_score - control_score` , so "is v2 better or worse, and by how much" is a single, direct number instead of something the reader has to compute themselves.

Sketch `evaluate_canary` yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the coin-flip idea and the tools for routing and comparing. Intermediate explains why the routing has to happen per-request (not per-chunk) for the canary group to be a genuine sample, and shows the real syntax for recording and grouping results. Advanced adds what a real canary rollout can't skip — an explicit, applied promotion rule instead of two numbers a human has to judge by eye, and a minimum sample size check so a tiny canary group doesn't produce a false-confidence verdict.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty list called results

for each request in test_requests:
    if a random number < 0.2:
        version = "v2"
    else:
        version = "v1"
    score = run_eval_suite for that request, using that version's prompt
    add {"version": version, "score": score} to results

split results into v1_results and v2_results
print the average score for each group
```

Here's almost the whole thing — just try running it and reading it line by line:

```
import random

def route_and_score(test_requests, prompts):
    results = []
    for request in test_requests:
        if random.random() < 0.2:
            version = "v2"
        else:
            version = "v1"
        score = run_eval_suite(prompts[version])["score"]
        results.append({"version": version, "score": score})
    return results

def average_by_version(results, version):
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r["score"])
    return sum(matching) / len(matching)
```

Expected output, running `route_and_score` over 20 fake requests then printing both averages:
something close to `v1 avg: 0.6` and `v2 avg: 1.0` — exact numbers vary run to run since routing is random.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function route_and_score(test_requests, prompts, canary_share) -> list[dict]:
  results = []
  for each request:
    version = "v2" if a random draw is under canary_share, else "v1"
    result = run_eval_suite(prompts[version])
    record version, score, and a fake cost estimate in results
  return results

function summarize(results, version) -> dict:
  matching = only the entries for this version
  return count, average score, average cost
```

```
import random

def route_and_score(test_requests: list, prompts: dict, canary_share: float = 0.2) -> list[dict]:
  results = []
  for request in test_requests:
    if random.random() < canary_share:
      version = "v2"
    else:
      version = "v1"
    eval_result = run_eval_suite(prompts[version])
    results.append({
      "version": version,
      "score": eval_result["score"],
      "cost": len(prompts[version]) * 0.0001, # stand-in cost estimate
    })
  return results

def summarize(results: list[dict], version: str) -> dict:
  matching = []
  for r in results:
    if r["version"] == version:
      matching.append(r)

  scores = []
  costs = []
  for r in matching:
    scores.append(r["score"])
```

```
        costs.append(r["cost"])

    return {
        "version": version,
        "count": len(matching),
        "avg_score": sum(scores) / len(scores),
        "avg_cost": sum(costs) / len(costs),
    }
```

Now call `summarize` for both `"v1"` and `"v2"` and print both, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function evaluate_canary(control_summary, canary_summary, min_sample_size) -> str
  if canary_summary["count"] < min_sample_size:
    return "insufficient data"

  score_gap = canary_summary["avg_score"] - control_summary["avg_score"]
  cost_ratio = canary_summary["avg_cost"] / control_summary["avg_cost"]

  if score_gap >= 0 and cost_ratio <= 1.2:
    return "promote"
  return "hold"
```

Here's almost the whole thing — fill in the printed verdict message yourself:

```
def evaluate_canary(control_summary: dict, canary_summary: dict, min_sample_size:
  if canary_summary["count"] < min_sample_size:
    return "insufficient data"

  score_gap = canary_summary["avg_score"] - control_summary["avg_score"]
  cost_ratio = canary_summary["avg_cost"] / control_summary["avg_cost"]

  if score_gap >= 0 and cost_ratio <= 1.2:
    return "promote"
  return "hold"

# your turn: call route_and_score with enough test requests that the
# canary group clears min_sample_size, build both summaries with
# summarize(), call evaluate_canary(), and print a verdict message
# that names the score gap and cost ratio, not just the one-word result
```

Fill in the calling code yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (route randomly, record, average) at 3 completeness levels — Basic just prints two raw averages for a human to compare by eye, Intermediate adds a `canary_share` parameter, a cost estimate, and a real `summarize()` function with full type hints, and Advanced turns the comparison into an actual applied decision — `evaluate_canary()` — with an explicit promotion rule and a minimum sample size guard, instead of leaving the "is this safe to promote" judgment call to whoever's reading the printout.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)